



Frameworks

...

Ingeniería de Software

M.TI. Juan Luis Campos Barrera

¿Qué es un Framework?

“Es un entorno de trabajo que ofrece una estructura base para desarrollar una aplicación. Proporciona un conjunto de herramientas, librerías, y convenciones que dictan cómo debe organizarse el código y cómo fluye la ejecución del programa”.

Origen del Framework

El concepto de “framework” en software nació de una necesidad fundamental: *dejar de reinventar la rueda.*

Los desarrolladores se dieron cuenta de que, para construir aplicaciones (especialmente las web), siempre resolvían los mismos problemas una y otra vez: gestionar peticiones, conectar a bases de datos, manejar plantillas, etc.

Un framework busca empaquetar estas soluciones comunes en una estructura reutilizable.

Breve de historia

Es difícil señalar un único “primer framework”, pero el concepto moderno tiene sus raíces en el patrón *MVC* (Modelo-Vista-Controlador), introducido en el lenguaje de programación Smalltalk-80 en la década de 1970.

MVC fue la primera formalización exitosa de la idea de separar las responsabilidades en una aplicación: los datos (Modelo), la interfaz de usuario (Vista) y la lógica de modelos (Controlador).

Algunos de los pioneros fueron: *ColdFusion* (1995) y *ASP* (Active Server Pages 1996)

Frameworks la aceptación

A principios del 2000 fue la “edad de oro” que definió el desarrollo web moderno. La popularización de lenguajes dinámicos y la necesidad de crear aplicaciones más complejas impulsaron una nueva generación de herramientas.

Framework	Año	Lenguaje
ColdFusion	1995	Adobe
ASP	1996	VBScript
Struts	2000	Java
Ruby on Rails	2004	Rails
Django	2005	Python

1996

A LA FECHA

2024

1996-2005



2006-2015



2016-2024



La era Moderna: el panorama actual es mucho más diverso y especializado

Frontend

La necesidad de crear interfaces de usuario ricas e interactivas en el navegador (Single-Page Applications o SPAs) provocó una explosión de frameworks de JavaScript.

AngularJS (2010): Creado por Google, trajo la arquitectura MVC/MVVM al frontend de una manera muy potente.

React (2013): La librería de Facebook (que se usa como un framework) que popularizó la arquitectura basada en componentes y el Virtual DOM, optimizando enormemente el rendimiento de las actualizaciones de la UI.

Vue.js (2014): Creado por un ex-desarrollador de Google, Evan You, buscaba ser una alternativa más ligera, flexible y progresiva a los grandes monolitos como Angular.

Backend

Los frameworks de backend también han evolucionado.

Node.js y Express.js (2010): Node.js permitió ejecutar JavaScript en el servidor. Express.js se convirtió rápidamente en el micro-framework estándar para Node, conocido por su minimalismo y flexibilidad.

Microframeworks: Surgió una tendencia hacia frameworks más pequeños y modulares como Flask (Python) y Sinatra (Ruby), que ofrecen solo lo esencial y dejan que el desarrollador elija el resto de las herramientas.

Nuevos Lenguajes: Lenguajes más modernos y de alto rendimiento como Go (Gin, Echo), Rust (Actix, Rocket) y Kotlin (Ktor) tienen sus propios ecosistemas de frameworks en crecimiento.

¿Porqué usar un Framework?

1. **Productividad:** Nos ahorran escribir código repetitivo y común (gestión de rutas, acceso a bases de datos, seguridad básica), permitiéndonos centrarnos en la lógica de negocio que hace única a nuestra aplicación. Es no "reinventar la rueda".
2. **Buenas Prácticas y Seguridad:** Los buenos frameworks están diseñados siguiendo *principios de arquitectura sólidos* y suelen incluir protección contra vulnerabilidades comunes (como SQL Injection o XSS). Te "guían" para que escribas un código más limpio y seguro.
3. **Estandarización** : Al imponer una estructura, hacen que los proyectos sean más fáciles de entender para otros desarrolladores que conozcan el framework, facilitando la colaboración y el mantenimiento a largo plazo.

¿Qué no es un Framework?

- Librerías:
 - request
 - oauth
 - qrcode
 - fpdf
 - etc
- CMS (Gestores de Contenido o Content Management System):
 - Wordpress
 - Joomla
 - Drupal
 - Shopify
 - etc

Los Favoritos

Django

- Python
- MVT
- Fullstack



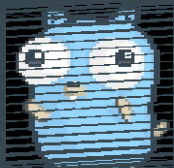
Flutter

- Dart
- Frontend UI
- Multiplataforma



Gin Gonic

- Lenguaje: Go
- Compilado
- Microservicios.



Temario

- Introducción
- Descripción MVC/MVT
- Requerimientos del sistema
- Django
 - Ambiente de trabajo y Creación de Proyecto
 - Estructura del proyecto
 - Comandos Básicos
 - Hola Mundo
 - Modelo
 - Vista
 - Template
 - Panel de Administración

Temario

...

- Django
 - Forms
 - Django Rest Framework
 - Comandos avanzados
 - Dockerización de Proyecto
- Gin Gonic
 - Enrutamiento/Manejo de Solicitudes
 - Templates y Renderizado
 - Middlewares
 - Conexión con Bases de Datos
 - Compilación
 - Dockerizacion de Proyecto

Temario

...

- Flutter
 - Ambiente de trabajo/ Herramientas de Desarrollo
 - Widgets
 - Layouts
 - Interacción con el usuario
 - Navegación
 - Conectividad y Datos
 - Consumo de APIs

Metodología del curso

Filosofía: Aprender haciendo (Learning by Doing)

- 80% Práctico: Dedicado a la codificación, resolución de problemas, desarrollo de proyectos y trabajo en equipo.
- 20% Teórico: Enfocado en la introducción de conceptos, arquitecturas, patrones de diseño y buenas prácticas que sustentan el desarrollo práctico.

Código de Conducta

- Llegar puntual, si llega tarde entrar en silencio
- Respetar a compañeros y profesor
- Evitar entregar tareas clonadas
- No copiar en examen
- Evitar el uso de celular
- No jugar en clase

Requirements

- Programación POO
- Conocimiento de Python
- Control de versiones Git
- Equipo portatil
- Ganas de aprender

Evaluación

Criterio	Porcentaje
Proyecto Django	40 %
Proyecto Go	20 %
Proyecto Flutter	10 %
Asistencia	10 %
<i>Proyecto Final (Equipo)</i>	<i>20 %</i>
<i>Calificación Final</i>	<i>100 %</i>

Código de conducta

- Llegar puntual, si llega tarde entrar en silencio
- Respetar a compañeros y profesor
- Evitar entregar tareas clonadas
- No copiar en examen
- Evitar el uso de celular
- No jugar en clase

Acerca de

M.T.I. Juan Luis Campos Barrera

Developer, ~~SRE~~, ~~Sys Admin~~, Docente....

Intereses Profesionales:

Linux, Python, IA, DevOps, Seguridad Informática, Open Source, etc

Intereses Personales:

Trail Running, Fotografía, Música, Comida ...

Presentación Individual

- Quién Soy?
- Qué hago aquí? (pregunta filosófica)
- Qué espero de la materia?
- Mis intereses personales... (opcional)

